

1

CODING AGENT — BIG PICTURE

Goal, What It Is, and Overall Architecture



GOAL

Build an autonomous coding assistant that understands natural language requests, uses tools to explore and modify code, runs commands, fixes issues, and delivers working results.



WHAT IS A CODING AGENT?

It is an AI agent powered by a large language model (LLM) that can reason, plan and take actions in your codebase using tools to accomplish complex software tasks with minimal human intervention.



Understands your request



Uses tools to interact with the environment



Observes results and adapts



Delivers the final outcome

KEY CAPABILITIES



Understand codebases

Read files, search, and build context



Modify code safely

Create, edit, or refactor files



Execute commands

Run tests, linters, builds, scripts



Debug and fix issues

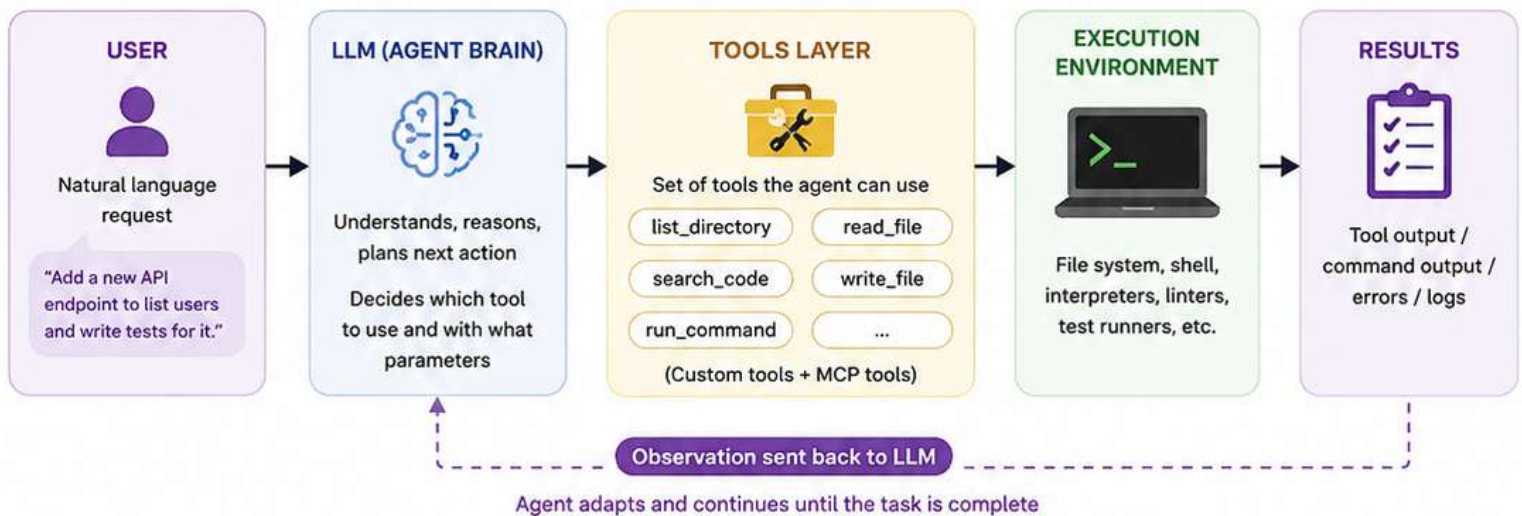
Analyze errors and iterate to fix them



Plan and reason step-by-step

Break down tasks and track progress

OVERALL ARCHITECTURE



WHY BUILD A CODING AGENT?



Automates repetitive development tasks



Increases productivity and focus



Consistent, accurate, and explainable actions



Learns and adapts through feedback

WHAT MAKES IT AN AGENT?



Autonomous: decides what to do next



Goal-driven: completes the task end-to-end



Tool-using: can interact with real systems



Iterative: observes results and improves



Context-aware: remembers conversation and code context



IN ONE LINE

A coding agent is an AI teammate that understands your goal, uses tools to work in your codebase, and iterates until it delivers results.



2 AGENT CORE LOOP

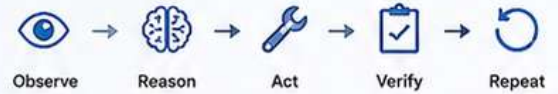
The Heart of a Coding Agent — Observe, Decide, Act, Verify, Repeat



CORE IDEA

The agent runs in a continuous loop where the LLM decides the next best action. Your application executes that action using tools and returns the result. The LLM observes the result and decides what to do next.

CLOSED-LOOP PRINCIPLE



USER REQUEST

User provides a natural language request.
"Fix the failing login tests."



LLM (DECIDE)

The LLM understands the context and decides the next action. It may call a tool or give a final answer.

WHAT FLOWS IN THE LOOP?

1. Context (conversation + memory)
2. Tool call (name + arguments)
3. Tool result (output / error / data)
4. Decision for next step

FINAL ANSWER

When the task is complete, the LLM returns a final answer / summary. Loop ends.

TERMINATION CONDITIONS

- Task completed successfully
- LLM returns a final answer
- Maximum iteration limit reached
- User stops the session
- Safety/guard rule triggered

REPEAT UNTIL TASK COMPLETE



LLM OBSERVES

LLM reads the tool result, updates its understanding, and decides the next step. Continue the loop.



TOOL RESULT

The result (output, data, logs, error, list of files, etc.) is returned to the LLM. This becomes new information for the next decision.



EXECUTE TOOL

Your application executes the tool in a controlled environment (file system, shell, tests, git, etc.). This is REAL action.



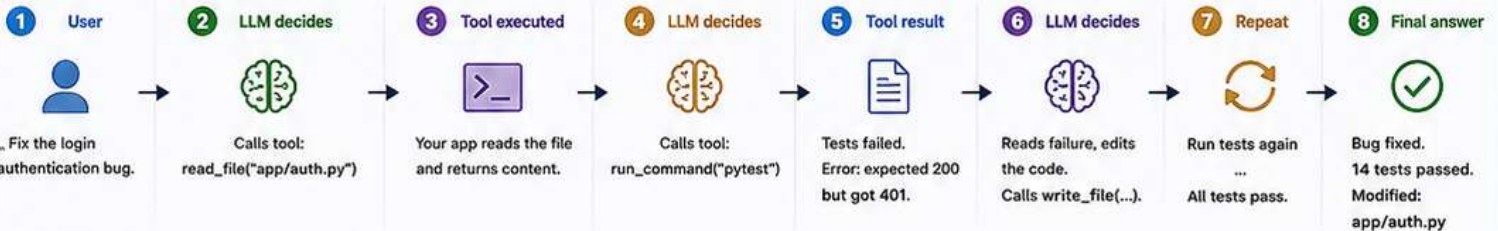
TOOL CALL

The LLM requests a tool with arguments.

Example tool call:

```
{
  "name": "read_file",
  "arguments": { "path": "app/auth.py" }
}
```

EXAMPLE: FIX A FAILING TEST



WHY THIS LOOP IS POWERFUL



Autonomous
LLM chooses the best next action.



Adaptive
Adjusts based on real results.



Verifiable
Actions are tested and confirmed.



Safe
You control what tools can do.



Effective
Solves complex tasks step by step.

PSEUDOCODE

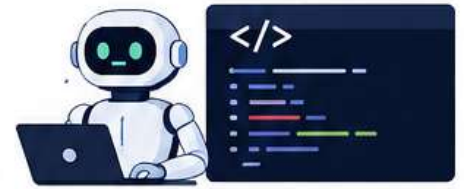
```
while not finished:
    response = llm(input, history)
    if response.has_tool_call:
        result = execute_tool(response.tool_call)
        history.append(result)
        continue
    else:
        return response.final_answer
```

REMEMBER The LLM is the brain. Your code is the body. Tools are the hands. Together they form an autonomous coding agent.



The Hands of Your Agent

Tools are functions your agent can call to interact with the codebase and the environment. They extend the LLM's abilities into the real world.



CORE TOOL CATEGORIES



1. FILE SYSTEM TOOLS

Explore and manipulate files in the workspace.



list_directory(path)
List files and folders in a directory.



read_file(path)
Read and return the content of a file.



write_file(path, content)
Create or overwrite a file with content.



get_file_info(path)
Get metadata (size, type, modified time).



2. SEARCH & DISCOVERY TOOLS

Find relevant code quickly.



search_code(query)
Search for text / symbols across the codebase.



find_symbol(symbol)
Find where a function/class/variable is defined.



grep(pattern, path)
Regex search within files.



3. EDITING TOOLS

Modify code safely.



write_file(path, content)
Create or overwrite entire file.



apply_patch(file, patch)
Apply unified diff / patch to a file.



insert_text(path, line, text)
Insert text at a specific line.



replace_text(path, old, new)
Replace text in a file.



4. EXECUTION TOOLS

Run commands and get results.



run_command(cmd)
Run shell command and return stdout, stderr, exit_code.



run_python(file)
Execute a Python file.



run_tests(cmd)
Run tests (pytest, unittest, etc.).



5. GIT TOOLS

Inspect repository state.



git_status()
Get repository status.



git_diff(path=None)
Get current diff (or for a file).



git_log(n=10)
Get recent commit history.



git_show(sha)
Show details of a commit.



6. UTILITY TOOLS

Helpful supporting utilities.



open_url(url)
Open a URL in browser (optional).



get_current_time()
Get current date and time.



calculate(expression)
Evaluate a simple expression.

HOW A TOOL IS USED IN THE LOOP



TOOL SCHEMA (WHAT THE LLM SEES)

Tools are described using a schema (JSON). This tells the LLM how to call them.

```
{
  "type": "function",
  "function": {
    "name": "read_file",
    "description": "Read and return the content of a file.",
    "parameters": {
      "type": "object",
      "properties": {
        "path": {
          "type": "string",
          "description": "File path"
        }
      },
      "required": ["path"],
      "additionalProperties": false
    }
  }
}
```

Same tool
Two sides

TOOL IMPLEMENTATION (WHAT YOUR CODE RUNS)

You implement the actual Python function.

```
def read_file(path: str) -> dict:
    try:
        with open(path, "r", encoding="utf-8") as f:
            content = f.read()
        return {
            "path": path,
            "content": content,
            "success": True
        }
    except Exception as e:
        return {"path": path, "error": str(e), "success": False}
```

TOOL REGISTRY (EXAMPLE)

Map tool names to schemas and implementations.

Tool Name	Category	Description	Implementation
list_directory	File System	List files in a directory	list_directory()
read_file	File System	Read file content	read_file()
write_file	Editing	Write or overwrite a file	write_file()
search_code	Search	Search text in codebase	search_code()
run_command	Execution	Run shell command	run_command()
run_tests	Execution	Run tests and return result	run_tests()
git_status	Git	Get git repository status	git_status()

💡 This registry is used by your agent loop to execute tool calls.

DESIGN PRINCIPLES

- 🎯 **Minimal but powerful**
Start with a small set of high-value tools.
- 💬 **Clear inputs & outputs**
Use simple arguments and structured results.
- 🛡️ **Safe by default**
Restrict paths to workspace and sensitive actions.
- ✅ **Deterministic & reliable**
Tools should be predictable and return accurate results.
- 🔧 **Extendable**
Easy to add new tools (local or MCP) later.



TASK START

The user gives a software development request. The agent completes it using tools in a closed loop.

"Fix the failing login tests in this project."

AGENT WORKFLOW (Step by Step)

1

EXPLORE PROJECT

Agent inspects the codebase to understand the structure.

```
> list_directory(".")
project/
├─ app/
├─ tests/
└─ README.md
```

2

UNDERSTAND THE PROBLEM

Agent searches and reads relevant files to find the issue.

```
> search_code("login")
app/auth.py
app/routes/login.py
tests/test_auth.py
```

3

ANALYZE CODE

Agent reads the files, understands the logic and identifies the bug.

```
> read_file("app/auth.py")
def authenticate(user, pwd):
    if user == "admin" and \
        pwd == "pass":
        return True
    return False
```

4

MODIFY / WRITE CODE

Agent edits the code with the required fix.

```
> write_file("app/auth.py")
def authenticate(user, pwd):
    if user == "admin" and \
        pwd == "password":
        return True # fixed
    return False
```

5

RUN TESTS / VERIFY

Agent runs tests to verify the change.

```
> run_command("pytest")
===== test session =====
FAILED tests/test_auth.py::test_login
E AssertionError: expected 200 but got 401
```

6

OBSERVE FAILURE

Agent reads the failure, understands why it failed.

```
E AssertionError:
E assert 401 == 200
E + where 401 = response.status_code
```

7

ITERATE AND FIX AGAIN

Agent updates the code and re-runs tests until it passes.

```
> write_file(...)
> run_command("pytest")
===== 8 passed in 1.42s
```

8

FINAL ANSWER

Agent explains changes and reports the result.

Task Completed ✓

- Fixed authentication bug in `app/auth.py`
- All 8 tests passed
- 1 file modified
- Description: Changed hardcoded password

Why This Workflow Works?

- Understands real project context
- Uses tools to interact with the actual code
- Observes real execution results (tests, logs)
- Iterates automatically until verified (not just text)
- Produces working, testable code, not just fixes.
- Human-like software engineering

Typical Development Tasks It Can Handle

- Fix bugs and failing tests
- Implement new features
- Refactor code
- Add / update APIs
- Write and update unit tests
- Improve documentation
- Set up small projects
- Code review and suggestions
- Run build / lint / format
- Analyze logs and error messages

Example Task Execution Trace



User

"Fix the failing login tests in this project."



Agent: Inspecting project structure...

```
> list_directory(".")
Found: app/, tests/, README.md
```



Agent: Searching for authentication code...

```
> search_code("login")
Found: app/auth.py, tests/test_auth.py
```



Agent: Reading app/auth.py ...

```
> read_file("app/auth.py")
```



Agent: Found hardcoded password bug. Updating file...

```
> write_file("app/auth.py")
```



Agent: Running tests...

```
> run_command("pytest")
FAILED: expected 200 but got 401
```



Agent: Investigating failure and fixing logic...

```
> read_file("app/routes/login.py")
> write_file("app/routes/login.py")
> run_command("pytest")
```



Agent: Tests passed! Task is complete.

✓ 8 tests passed · 1 file modified

The authentication bug was caused by a hardcoded password and missing token validation. I updated `auth.py` and `login.py` and verified the fix with tests.

Workflow Diagram (Closed Loop)



4 CODING AGENT — AUTONOMOUS WORKFLOW

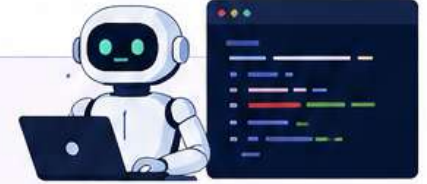
From Request to Verified Solution



WHAT DOES THE AGENT DO?

The agent solves real software tasks end-to-end using a loop:

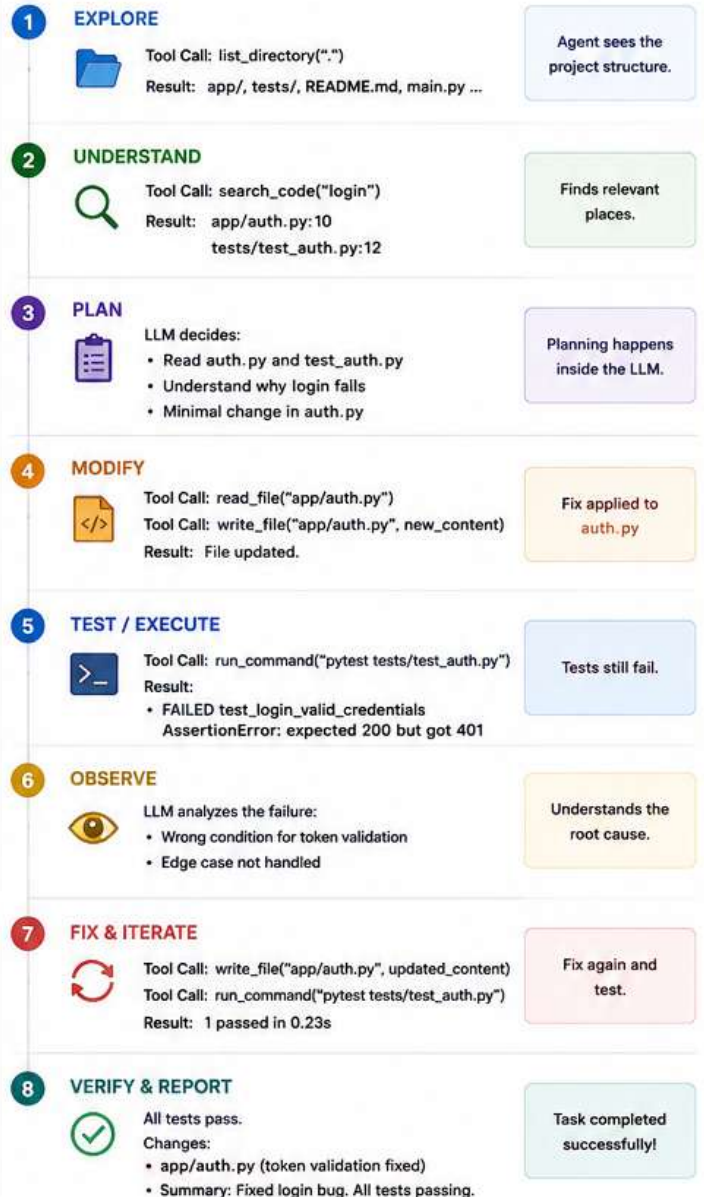
Explore → Understand → Plan → Modify → Test → Observe → Fix → Verify → Report



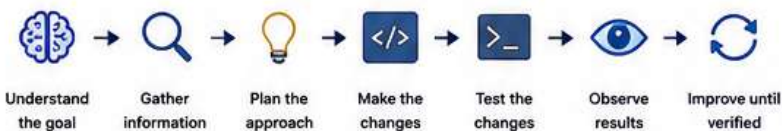
THE WORKFLOW (HIGH LEVEL)



EXAMPLE: FIX A FAILING TEST



SMART STRATEGY OF THE AGENT



TYPES OF TASKS IT CAN HANDLE

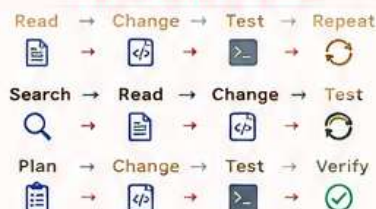
- ✓ Fix bugs and failing tests
- ✓ Refactor and improve code
- ✓ Add new features
- ✓ Update dependencies
- ✓ Improve documentation
- ✓ Optimize performance



GOLDEN RULES FOR THE AGENT

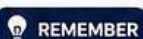
- ★ Understand before changing – Read, search, and think first.
- ★ Make minimal changes – Do only what is necessary.
- ★ Test after every important change – Never assume it works.
- ★ Observe and learn – Use failures to guide the next step.
- ★ Verify before finishing – All tests must pass (or task is validated).
- ★ Be safe – Respect workspace and permissions.

COMMON LOOP PATTERNS



SUCCESS INDICATORS

- ✓ Requested task achieved
- ✓ All tests pass
- ✓ Changes are minimal & correct
- ✓ Code quality maintained
- ✓ Clear summary provided
- ✓ User satisfied



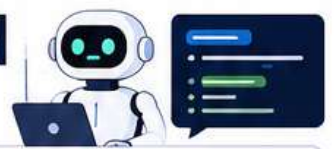
REMEMBER

The workflow is not fixed. The agent adapts based on the project and the problem. It thinks, acts, learns, and iterates — until the job is done.



5 AGENT STATE & TOOL COMMUNICATION

How the Agent Remembers, Talks, and Learns



CORE IDEA

The agent maintains conversation state (memory) and communicates with the LLM using tool calls and tool results. This back-and-forth enables reasoning, action, and learning from real-world feedback.

THE COMMUNICATION CYCLE



1. CONVERSATION STATE (MEMORY)

You maintain the history locally and send it to the LLM every time.

Example History



In Code (messages list)

```
messages = [
  {"role": "user",
   "content": "Fix the login bug."},
  {"role": "assistant",
   "content": [
     {"type": "function_call",
      "name": "list_directory",
      "arguments": { "path": "." },
      "call_id": "call_1" }
   ]},
  {"role": "tool",
   "content": "{ \"entries\": [ \"app\", \"tests\", \"README.md\" ] }",
   "tool_call_id": "call_1"},
  ...
]
```

The more accurate the history, the better the agent's decisions.

2. TOOL COMMUNICATION FORMAT

The LLM requests tools using function-calling.

A. TOOL CALL (from LLM)

```
{
  "type": "function_call",
  "name": "read_file",
  "arguments": { "path": "app/auth.py" },
  "call_id": "call_42"
}
```

- name: tool to call
- arguments: JSON object
- call_id: unique id for matching result

B. TOOL RESULT (from your app)

```
{
  "type": "function_call_output",
  "call_id": "call_42",
  "output": {
    "path": "app/auth.py",
    "content": "def login(user, pwd):\n ...",
    "success": true
  }
}
```

Send this result back in the next request. The LLM uses it to reason and decide the next step.

3. TOOL RESULT DESIGN (RETURN STRUCTURED DATA)

Always return structured, informative, and consistent results.



★ Structured results reduce ambiguity and help the LLM make better next decisions.

4. STATE FLOW EXAMPLE (BEHIND THE SCENES)

Step	Who	What happens	Example
1	User	Provides the request	Fix the login bug.
2	LLM	Decides to call a tool	read_file("app/auth.py")
3	Your App	Executes the tool	Reads file from disk
4	Your App	Returns the result	{ "path": "...", "content": "..." }
5	LLM	Analyzes result & decides next action	run_command("pytest")
6	Your App	Executes command	Runs tests
7	Your App	Returns output	{ "exit_code": 1, "stderr": "..." }
...	...	Loop continues until task completed	

This loop creates continuous context + action + feedback.

5. HOW THE LLM REASONS WITH STATE

LLM sees:

History + Results
(User request, tool calls, results, outputs, errors, etc.)

LLM reasons:

- Understands the situation
- Identifies what's missing
- Plans the next action
- Decides which tool to use



LLM acts:

Calls a tool or returns the final answer

Example reasoning pattern

- See failing test output (AssertionError on token validation)
- Infer root cause area (likely condition in auth.py)
- Plan fix (read file → modify condition → save)
- Verify fix (run tests again)
- If still failing → iterate Else → provide final report

Key Point
State is the bridge between information (context) and action (tools).

6. BEST PRACTICES

- Keep history clean and relevant (trim long histories).
- Return structured, machine-readable results.
- Use clear, consistent tool names and arguments.
- Always include success/failure and meaningful details.
- Limit tool output size; summarize if needed.
- Log everything for debugging and improvement.

7. WHAT NOT TO DO

- Don't return unstructured or vague text.
- Don't hide errors—return exact error output.
- Don't give the LLM too many tools at once.
- Don't ignore call_id matching.
- Don't let history grow without limits.



SUMMARY

Your agent is a closed feedback system:

State → Tool Call → Execution → Result → State → ...
= Smart, adaptive, and reliable behavior.

★ Good state management + clear tool communication = powerful coding agent.



REMEMBER

- The LLM is the brain.
- Tools are the hands.
- State is the memory.
- Together, they build intelligence in action.



Brain (LLM)



Tools (Hands)



State (Memory)



Intelligent Agent

6

CODING AGENT – SAFETY & CONTROL

Keep the Agent Powerful, but Always in Your Control



WHY SAFETY MATTERS

Agents can read, write, and execute. Without guardrails, mistakes can damage data, leak secrets, or affect your system. Safety enables trust, reliability, and real-world use.

SAFETY PRINCIPLES



Least Privilege
Only allow what is necessary.



Bounded Environment
Operate inside a safe workspace.



Visibility
Log actions and show everything.



Human in the Loop
Require approval for risky actions.



Fail Safe
Prefer blocking over allowing.

CORE SAFETY MECHANISMS

1 WORKSPACE RESTRICTION



Allowed Workspace
/workspace/project
|-- app/
|-- tests/
|-- docs/
|-- README.md

- Agent can access only files inside the workspace.
- Prevent path traversal (e.g., ../../etc/passwd).
- Enforce absolute path checks in every tool.

✓ **Allowed:** /workspace/project/app/auth.py
✗ **Blocked:** /etc/passwd, /home/user/.ssh/id_rsa

2 ITERATION LIMIT & TIMEOUTS



- Set a maximum number of iterations (e.g., 20).
- Set per-command timeout (e.g., 60s).
- Stop the loop if limits are reached.

Example
MAX_ITERATIONS = 20
COMMAND_TIMEOUT = 60s

3 DANGEROUS COMMAND CONTROL



Block or require approval for high-risk commands.

Examples (requires approval)

```
rm -rf /  
dd if=/dev/zero of=/dev/sda  
{:}{:}{:}&:::  
shutdown -h now  
git push --force
```



Allow harmless commands automatically (pytest, ls, cat, python ...).

4 FILE OPERATION GUARDRAILS



Write Protection
Block writes to sensitive paths (e.g., .env, .git, system files).



Binary / Large File Protection
Block writing binary files or extremely large files.



Deletion Control
Require approval before deleting files or folders.

Protected Patterns (example)
.env, .git/*, *.key, *.pem, /etc/*, /bin/*
*.png, *.jpg (by default read-only)

5 HUMAN APPROVAL LAYER



For risky actions, ask the user before executing.

Example Approval Request

The agent wants to execute:
git push origin main
This may modify remote repository.
Allow this action? (y/n)



You can also approve at tool level (MCP approval style).

6 AUDIT & LOGGING



Log every important event.

- Tool calls
- Inputs & outputs
- Files read/written
- Commands executed
- Approvals / rejections

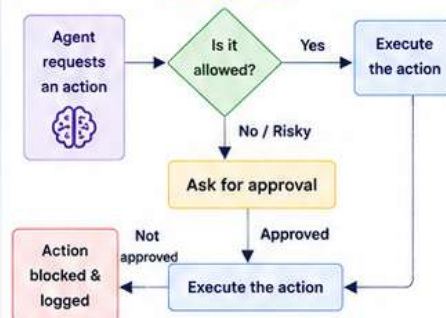
Log Entry (example)

```
{  
  "timestamp": "2025-05-17T10:30:15Z",  
  "type": "run_command",  
  "command": "pytest",  
  "exit_code": 1,  
  "approved": true  
}
```

RISK LEVEL CLASSIFICATION (EXAMPLE)

Risk Level	Examples	Action
Low	read_file, list_directory, search_code, get_file_info	Auto allow
Medium	write_file, run_tests, pip install, git diff	Auto allow (with logging)
High	run_command (arbitrary), delete_file, git commit	Require approval
Critical	rm -rf, format disk, shutdown, git push --force	Block by default (must approve)

SAFETY FLOW



WHAT THIS PROTECTS AGAINST

- 🗑️ Accidental data loss or deletion
- 🦋 Malicious prompt injection attempts
- 🔑 Secrets or key leakage
- ♾️ Infinite loops and resource abuse
- 🔧 Unsafe system modifications

✓ **Safety is not about limiting power. It is about using power responsibly.**

REMEMBER



Start restrictive. Expand permissions gradually.



Make the agent's actions observable and transparent.



Keep humans in the loop for risky decisions.



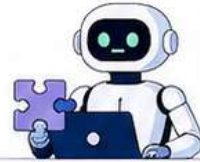
Log everything. It helps debug and build trust.



Safe agent today, autonomous engineer tomorrow.

CODING AGENT — MCP INTEGRATION

Extend Your Agent Beyond Local Tools



MCP = Model Context Protocol
A standard way for LLMs to discover and use external tools and services.



WHAT IS MCP?

MCP is an open protocol that lets your agent connect to external servers (tools, APIs, data sources, services) in a standardized, secure, and extensible way.

WHY MCP?

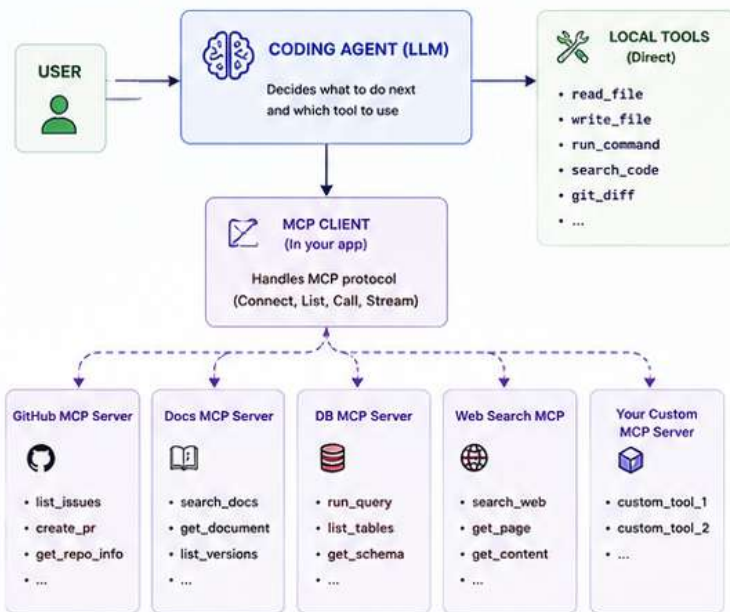
 Extend capabilities without rewriting code

 Interoperate with many tools & services

 Standardized & future-proof

 Ecosystem of MCP servers

ARCHITECTURE: CODING AGENT WITH MCP



LOCAL TOOLS vs MCP TOOLS

LOCAL TOOLS

- ✓ Run inside your application
- ✓ Direct function calls
- ✓ No network latency
- ✓ Best for file system, shell, and simple operations
- ✓ Full control and security

VS

MCP TOOLS

- ✓ Run on external MCP servers
- ✓ Communicate via MCP protocol (JSON-RPC over stdio / HTTP)
- ✓ Extensible ecosystem
- ✓ Best for external systems, APIs, data, services
- ✓ Need authentication/approvals sometimes

EXAMPLES OF MCP SERVERS FOR CODING AGENT



MCP CALL FLOW (High Level)



MCP PROTOCOL BASICS

 Transport	stdio, HTTP(S), WebSocket (depends on server)
 Format	JSON-RPC 2.0
 Core Methods	• initialize – handshake • tools/list – list available tools • tools/call – call a tool • tools/stream – stream results (optional)
 Security	Auth, scopes, approvals, encrypted channels (when needed)

APPROVALS WITH MCP

Some MCP servers or tools may require human approval before executing sensitive actions.

Example Approval Request
Tool: github.create_pull_request
Repository: user/repo
Title: Fix login bug
Branch: fix/login
Allow this action?

You can define rules for auto-approve, deny, or always ask.

WHEN TO USE MCP?

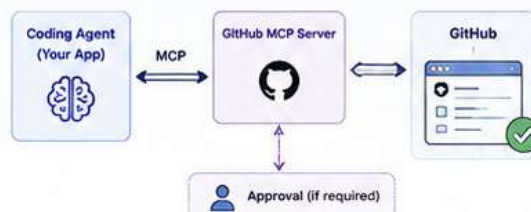
- ✓ Need data from external systems (GitHub, DB, APIs)
- ✓ Need specialized capabilities (search, deployments, cloud)
- ✓ Want to reuse community MCP servers
- ✓ Want to build a modular, enterprise-ready agent

 Start with local tools.
Add MCP when you need more power.



END-TO-END EXAMPLE: CREATE GITHUB PR WITH AGENT

- 1 User: "Fix the bug and create a PR."
- 2 Agent reads code, fixes, runs tests (local tools).
- 3 Agent runs git diff (local tool).
- 4 Agent decides to create a PR.
- 5 Uses MCP: github.create_pull_request
- 6 MCP server creates the PR.
- 7 Agent reports PR link to the user.



BEST PRACTICES

- ✓ Keep sensitive tools behind approval.
- ✓ Use least-privilege tokens and scopes.
- ✓ Validate and sanitize all inputs.
- ✓ Log all MCP calls and results.
- ✓ Handle errors, timeouts, and retries.
- ✓ Design your agent to work even if an MCP server is unavailable.



KEY TAKEAWAY

MCP is how your coding agent goes beyond its local environment. It connects to the world — safely, securely, and in a standard way.



Coding Agent (Brain)



Local Tools (Hands)



MCP Tools (Outside World)



Powerful Autonomous Developer



PROJECT GOAL

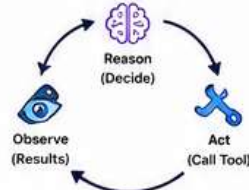


Build a terminal (later TUI) based coding agent that can understand a request, explore a repository, modify code, run tests/commands, observe results and iteratively fix problems until the task is completed and verified.

KEY CAPABILITIES

- ✓ Explore & understand codebase
- ✓ Read, search and reason about code
- ✓ Modify code safely
- ✓ Execute commands & run tests
- ✓ Observe results & iterate
- ✓ Report final outcome with changes

WHAT MAKES IT AN AGENT?



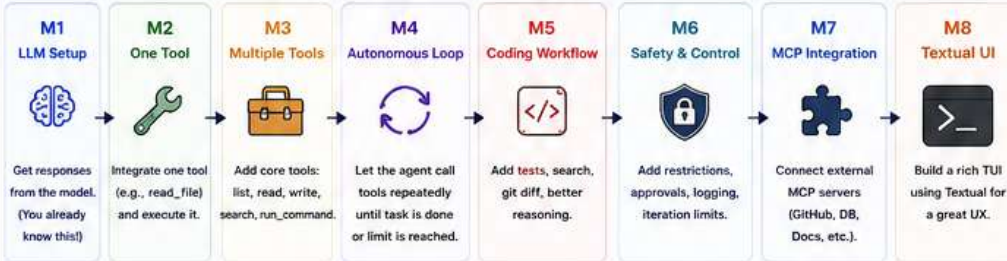
Repeat until goal is achieved or limit is reached

1. PROJECT STRUCTURE (SUGGESTED)



The agent operates ONLY inside workspace/target-project/

2. DEVELOPMENT MILESTONES (ROADMAP)



Remember:

- Start simple. Make the core loop rock solid.
- Add one capability at a time.
- Test with real projects and iterate.

3. WHAT THE AGENT DECIDES (EXAMPLES)

- Which files to inspect?
- Which tool to call next?
- What change to make?
- Which tests/commands to run?
- How to fix failures?
- When the task is done?



WHAT YOUR APP EXECUTES

```
read_file(...)
write_file(...)
run_command(...)
search_code(...)
git_diff(...)
... (and more tools)
```

LLM = Brain (Decision Maker) • Your App = Execution Engine

4. TOOL RESULT FORMATS (ALWAYS STRUCTURED)

read_file

```
{
  "path": "app/auth.py",
  "content": "def login(...): ...",
  "encoding": "utf-8",
  "lines": 120
}
```

search_code

```
{
  "query": "authenticate",
  "results": [
    {
      "path": "app/auth.py",
      "line": 18
    },
    {
      "path": "tests/test_auth.py",
      "line": 12
    }
  ],
  "count": 2
}
```

run_command

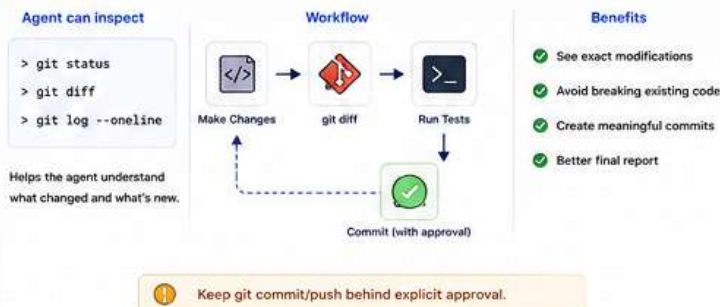
```
{
  "command": "pytest -q",
  "exit_code": 1,
  "stdout": "...",
  "stderr": "AssertionError: ...",
  "duration_ms": 1450
}
```

list_directory

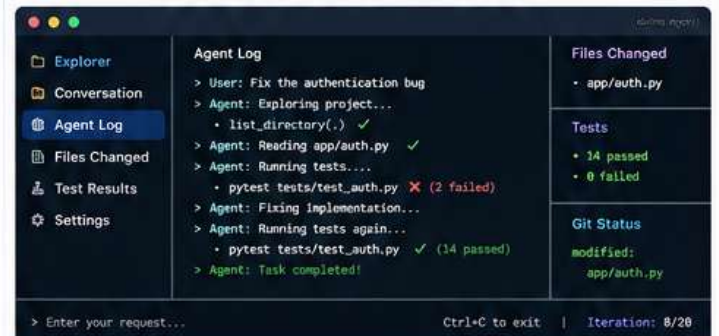
```
{
  "path": ".",
  "entries": [
    {
      "path": "app",
      "type": "directory",
      "count": 3
    },
    {
      "path": "tests",
      "type": "directory",
      "count": 3
    },
    {
      "path": "README.md",
      "type": "file",
      "count": 1
    }
  ]
}
```

Structured results reduce ambiguity and help the LLM reason better.

5. GIT INTEGRATION (POWERFUL CONTEXT)



6. TEXTUAL UI (FINAL EXPERIENCE PREVIEW)



7. FINAL ARCHITECTURE OVERVIEW



Closed Loop
Observe → Reason
→ Act → Verify

Extensible
Add new tools or
MCP servers easily.

Safe & Controlled
Workspace, approvals,
limits, and logging.

Verifiable
Test, observe and
report with evidence.

Production Ready
From terminal to beautiful
TUI with Textual.



KEY TAKEAWAY

You are building an autonomous system where the LLM plans and decides, and your code executes safely and reliably. With a solid loop, good tools, and strong safety — your agent can solve real coding tasks just like a junior developer.

